数字考古透视镜 — 三合一旗舰升级项目计划（v2）

> 2026-11-10 确定方案，2026-11-12 更新为 v2（核心档 + 加强档）。
> 纯前端演示工具升级为 AI 驱动的文化知识平台。

一、项目背景

现有项目"数字考古透视镜"是纯前端 HTML+CSS+JS 的编码可视化工具（含 Electron 桌面版），核心功能：

- 汉字三步变身动画（甲骨文 → 篆书 → 简体）
- 编码对比器（Unicode/UTF-8/UTF-16/GB2312/GBK/Big5）
- 原项目队友：汪思涵（视频剪辑）、成姿茹（说明书撰写）

本次升级目标：从纯前端工具升级为 **前端 + 后端 + AI** 三层合一的完整产品。

二、目标架构

层	技术栈	说明
前端	Vue3 + ECharts/D3 + CSS3动画	组件化重构，可视化交互
后端	Python FastAPI + PostgreSQL + Neo4j	REST API + 用户系统 + 知识图谱
AI层	RAG + DeepSeek API + LangChain + ChromaDB + V4.1 Flash OCR	智能问答 + 古文字识别

三、模块清单

🔴 核心档（必做）

前端（Phase 1）

模块	说明
透视镜三步变身动画	迁移为Vue组件，保持CSS3动画质量
编码对比器	Unicode/UTF-8/UTF-16/GB2312/GBK/Big5
汉字演变时间轴	D3实现，可交互
用户中心页面	UI骨架，Phase 2接后端
AI对话面板	UI骨架，Phase 3接RAG
知识图谱可视化	D3/ECharts力导向图，展示汉字关系网络
响应式布局	移动端适配

后端（Phase 2）

模块	说明
FastAPI项目骨架	路由结构、中间件、异常处理
PostgreSQL数据库	汉字知识表：汉字→编码信息→演变历史→字形图片
用户系统	JWT认证 + 注册登录 + 角色权限（普通用户/管理员）
汉字数据API	按笔画/部首/编码/朝代检索，CRUD管理
学习记录	用户收藏、浏览历史、笔记功能

AI层（Phase 3）

模块	说明
RAG文化导览问答	完整管道：数据源→向量化→检索→大模型生成→引用溯源
知识图谱数据	Neo4j/NetworkX构建汉字关系（同源/形近/通假/异体）
DeepSeek API接入	文化问答 + 汉字故事生成

整合部署（Phase 4）

模块	说明
前后端联调	API对接、跨域处理
Vercel部署前端	免费档
Render部署后端	免费档 + PostgreSQL
整体测试	功能验收、性能检查

🟡 加强档（核心档完成后追加）

模块	层	说明
OCR古文字识别	AI	用户上传甲骨文/金文图片，DeepSeek V4.1 Flash识别→返回现代汉字+编码
数据爬取模块	后端	Python爬虫从汉典/国学大师抓取汉字演变数据，定时任务更新

⚪ 全量档（暂不考虑）

模块	原因
WebSocket实时功能	技术栈过重，收益有限
智能推荐算法	数据量不够，效果出不來
在线协作	超出学生项目范围

四、可行性评估

层	选型	可行性	备注
前端	Vue3 + ECharts/D3 + CSS3	✅很合适	

层	选型	可行性	备注
			现有透视镜动画可直接迁移，D3做时间轴和知识图谱
后端	Python FastAPI + PostgreSQL	✅推荐	AI层生态在Python侧，一套语言打通后端+AI
知识图谱	Neo4j 或 NetworkX	✅亮点	汉字关系网络可视化，技术含金量最高
AI	RAG + DeepSeek API	✅核心亮点	知识库放汉字编码/演变资料，检索增强生成
OCR	DeepSeek V4.1 Flash	✅可行	原生多模态视觉理解，按token付费成本低
数据	SQLite → PostgreSQL	✅合理	开发期用SQLite，上线切PostgreSQL
部署	Vercel + Render	✅免费可行	前端Vercel，后端Render免费档够用

五、关键决策

1. **后端选 FastAPI 而非 Node**：RAG 核心工具链（LangChain、ChromaDB、embedding）全在 Python，一套语言打通后端 + AI
2. **AI 模型用 DeepSeek API**：已有 key，成本低，中文能力够用
3. **知识图谱用 Neo4j 或 NetworkX**：Neo4j 更专业但部署重，NetworkX 轻量先用，后期可迁移
4. **OCR 用 V4.1 Flash 而非 V4 Pro**：V4 Pro 不支持图片识别，V4.1 Flash 原生多模态
5. **数据库开发期 SQLite，上线 PostgreSQL**：Render 提供免费 PostgreSQL（90天有效）
6. **前端 Vue3**：学习曲线平缓，适合可视化 + 内容型产品

六、执行顺序

```
Phase 1: 前端重构（1-2周）
 原生JS → Vue3组件化，保留动画，加路由和页面框架
  ├─ Vite搭建Vue3项目骨架
  ├─ 透视镜三步变身动画迁移为Vue组件
  ├─ 编码对比器迁移为Vue组件
  ├─ 汉字演变时间轴（D3实现）
  ├─ 知识图谱可视化页面（D3力导向图）
  ├─ 用户中心页面（UI骨架）
  ├─ AI对话面板（UI骨架）
  └─ 数据先用mock，为Phase 2预留接口

Phase 2: 后端搭建（1-2周）
  FastAPI项目结构 + 数据库 + 汉字数据API + 用户系统
  ├─ FastAPI项目初始化
  ├─ PostgreSQL数据库设计 + 汉字知识表
  ├─ REST API：汉字查询（笔画/部首/编码/朝代）
  ├─ JWT用户鉴权系统 + 角色权限
  ├─ 学习记录API（收藏/浏览历史/笔记）
  └─ 知识图谱数据接口（供前端D3消费）

Phase 3: AI层接入（1-2周）
 知识库构建 + RAG管道 + 知识图谱 + AI问答接口
  ├─ 汉字编码/演变知识库构建
  ├─ ChromaDB/FAISS向量索引
  ├─ LangChain RAG管道搭建（检索→生成→引用溯源）
  ├─ DeepSeek API接入（文化问答 + 汉字故事生成）
  ├─ Neo4j/NetworkX知识图谱构建（同源/形近/通假/异体关系）
  └─ AI问答接口 + 图谱查询接口

Phase 4: 整合 + 部署（3-5天）
 前后端联调 + Vercel/Render部署
  ├─ 前后端API联调
  ├─ 前端Vercel部署
  ├─ 后端Render部署
  ├─ PostgreSQL生产环境配置
  └─ 整体测试验收

Phase 5: 加强档（核心档完成后，有时间再追加）
  ├─ OCR古文字识别（DeepSeek V4.1 Flash多模态）
  │    ├─ 图片上传接口
  │    ├─ V4.1 Flash API调用（model: deepseek-flash）
  │    └─ 返回识别结果 + 对应现代汉字 + 编码信息
  └─ 数据爬取模块
       ├─ Python爬虫（汉典/国学大师）
       ├─ 数据清洗 + 去重
       └─ 定时任务更新数据库
```

七、部署方案

服务	用途	费用
Vercel	前端部署	免费档
Render	后端部署	免费档
Render PostgreSQL	数据库	免费（90天有效）
Neo4j Aura（可选）	知识图谱数据库	免费档（限1个实例）
DeepSeek API	AI模型调用	按量付费（已有余额）

八、技术约束

- Phase 1 前端只做UI和交互，数据先用mock
- 组件化、可维护，为Phase 2后端API对接预留接口
- 动画效果保持原项目质量
- 每个Phase完成后验证再进入下一个
- Claude Code 开发时一次专注一个模块，做完存文件，避免上下文溢出
- 知识图谱先用 NetworkX + JSON 输出，前端 D3 消费；后期如需可迁移 Neo4j

九、Claude Code 使用提示词

启动新对话时粘贴以下提示词：

```
我要继续开发"数字考古透视镜"升级版项目（Vue3 + FastAPI + RAG）。

当前项目状态：
- 项目目录：[填写你的项目路径]
- 旧代码在 legacy/ 目录下
- 已完成的模块：[填写当前进度]

请先读取项目目录结构和已有代码，然后帮我继续开发 [当前要做的新模块]。

技术栈：
```

```
- 前端：Vue3 + Vite + ECharts/D3 + CSS3
- 后端：Python FastAPI + PostgreSQL
- AI：LangChain + ChromaDB + DeepSeek API
- 知识图谱：NetworkX（后期可迁移Neo4j）

约束：
- 一次只做一个模块，做完保存文件
- 代码要有注释，组件化，可维护
- 数据先mock，预留后端接口
```

> 更新日期：2026-11-12
> 版本：v2（核心档 + 加强档）
> 原始版本：2026-11-10 v1

> 本内容由 Coze AI 生成，请遵循相关法律法规及《人工智能生成合成内容标识办法》使用与传播。